

Lighthouse Storage Integration for mindX

Doc status: authoritative integration guide for mindX core developers and for autonomous agents managed by mindX. Written against Lighthouse Storage docs, the [lighthouseweb3](#) Python SDK (v0.1.6, Oct 2025), [@lighthouse-web3/sdk](#) (v0.4.1), and [@lighthouse-web3/kavach](#) (v0.2.1) as of April 2026. **Audience:** mindX core devs wiring [mindx/storage/lighthouse_client.py](#) and agent authors who need persistent decentralized storage from inside the AgenticPlace / DELTAVERSE / PYTHAI ecosystem. **Scope:** account/key flows, all SDK and HTTP surfaces, encrypted Kavach uploads, Filecoin deal verification, IPNS, agent storage patterns, x402-metered uploads, BANKON / BONAFAIDE token-gated access, and a drop-in production Python module.

Why Lighthouse for mindX

mindX needs **content-addressed, durable, programmable storage** for agent memory, model weights, RAG corpora, POD checkpoints, and shared knowledge bases. Lighthouse uniquely fits because it is **the only IPFS-native service that funds long-term Filecoin deals from a smart-contract endowment pool** — a "pay once, store forever" protocol — instead of charging recurring rent like Pinata or relying on best-effort pinning like web3.storage. Each upload is content-addressed (the same bytes always yield the same CID), so mindX never double-pays for duplicated data and can use CIDs as cryptographic primary keys across BONAFAIDE reputation records, AlgoIDNFT-bound DIDs, and the chain mapping at

[agenticplace.pythai.net/allchain.html](#). [GitHub +4 +2](#)

Three architectural properties matter for mindX specifically. **First**, Kavach threshold encryption (3-of-5 across [encryption.lighthouse.storage](#)) gives agents wallet-gated private storage without trusting any single party — a clean fit with BANKON soulbound DIDs. **Second**, Lighthouse's documented **x402 pay-per-use upload** flow lets mindX meter agent storage spend through the **Algorand x402 rails via Parsec/parsec-wallet** without provisioning per-agent API keys. **Third**, the new **Lighthouse MCP server** ([lighthouse-agent-tooling](#)) exposes upload/fetch/dataset tools natively to autonomous agents — mindX agents can use it as an MCP tool out of the box. [Lighthouse +2 +4](#)

Lighthouse **complements** rather than replaces mindX's existing IPFS-only DEX layer: the same CIDs that flow through your IPFS DEX stack become Lighthouse-pinned and Filecoin-sealed, providing **trustless permanence with PoDSI proofs** for any artifact mindX wants to make auditable on-chain.

Quickstart

Get from zero to first upload in under five minutes.

Python (canonical for mindX)

bash

```
python -m pip install lighthouseweb3 requests eth-account pycryptodome
export LIGHTHOUSE_TOKEN="lh_xxx_paste_dashboard_key"
```

python

```
from lighthouseweb3 import Lighthouse

lh = Lighthouse(token=None) # reads env LIGHTHOUSE_TOKEN
res = lh.upload(source="./agent_state.json")
cid = res["data"]["Hash"]
print(f"Pinned: https://gateway.lighthouse.storage/ipfs/{cid}")
```

Node.js (for any feature the Python SDK lacks)

bash

```
npm i @lighthouse-web3/sdk @lighthouse-web3/kavach ethers
export LIGHTHOUSE_API_KEY="lh_xxx"
```

javascript

```
import lighthouse from "@lighthouse-web3/sdk";
const r = await lighthouse.upload("./agent_state.json", process.env.LIGHTHOUSE_API_K
console.log("CID:", r.data.Hash);
```

Reality check on the Python SDK. `lighthouseweb3` is intentionally narrow. It implements `upload`, `uploadBlob`, `getUploads`, `download`, `getDealStatus`, `getTagged` — and **nothing else**. Kavach encryption, IPNS, balance, file-info, programmatic API-key generation, share/revoke, and access conditions are **JS-only** in the SDK. mindX therefore uses a hybrid pattern: the Python SDK for plain uploads, raw HTTP via `requests` for everything else, and a Node-helper subprocess fallback for Kavach features that require Shamir + AES against the live encryption-node REST surface (the per-node REST paths are not all publicly documented). `Lighthouse`

Authentication and key management

Lighthouse uses **two distinct credentials** that mindX must keep separate.

1. The dashboard API key (`LIGHTHOUSE_API_KEY`, prefixed `lh_...`). It is the bearer token for `api.lighthouse.storage` and `upload.lighthouse.storage`. Generate it once at `https://files.lighthouse.storage/` (Login → API Key panel) or programmatically: `GET https://api.lighthouse.storage/api/auth/get_message?publicKey=<addr>` returns a nonce, sign it with the wallet's private key (EIP-191 `personal_sign`), then `POST https://api.lighthouse.storage/api/auth/create_api_key` with `{publicKey, signedMessage, keyName}`. **Never commit this key.** mindX should load it from environment, a Podman secret, or a vault entry — `os.getenv("LIGHTHOUSE_API_KEY")` only. (Lighthouse +5 +3)

2. The Kavach auth token (per-wallet signed message or JWT). Required for **every** call to `encryption.lighthouse.storage` (encrypted upload, share, revoke, decrypt, access conditions). Obtained via `GET https://encryption.lighthouse.storage/api/message/{publicKey}` → sign → optionally exchange for a JWT via `kavach.getJWT(addr, signed)`. The JWT is the recommended form for repeated calls so the agent's wallet is not asked to sign every single operation. (Lighthouse +3)

For agent scoping, mindX should **mint a derived API key per agent class** (one for memory writers, one for RAG curators, etc.) by signing the Lighthouse auth message with a per-class wallet derived from the BANKON master DID. This way usage and revocation are separable, and BONAFIDE can attribute storage spend back to specific agents without exposing the parent key. Treat Kavach signed messages as **agent-session credentials**: cache the JWT in memory only, never persist it to disk, and re-derive on restart. Both credentials should live in `.env` files that are listed in `.gitignore` and mounted into Podman containers via `--env-file` (never baked into images). (Lighthouse +2)

Core storage operations

This section covers every read/write path mindX agents will hit in production. Code is Python where the SDK supports it natively; HTTP-direct elsewhere.

Upload a file

```
python
```

```

from lighthouseweb3 import Lighthouse
lh = Lighthouse(token=None) # env LIGHTHOUSE_TOKEN
res = lh.upload(source="/var/lib/mindx/checkpoints/agent_42.bin", tag="agent-42-ckpt")
# {'data': {'Name': 'agent_42.bin', 'Hash': 'Qm...', 'Size': '12345'}}
cid = res["data"]["Hash"]

```

The same call accepts a directory path and recursively uploads it as a UnixFS DAG, returning the root CID. Tags are server-side labels you can later filter via `lh.getTagged(tag)`. [Lighthouse + 3](#)

Upload JSON for agent state

`lighthouseweb3` has no `uploadText` — use `uploadBlob` with an in-memory buffer. This is the canonical pattern for **agent memory snapshots**.

```

python

import io, json
from lighthouseweb3 import Lighthouse

def upload_json(payload: dict, name: str, tag: str = "") -> str:
    lh = Lighthouse(token=None)
    blob = io.BytesIO(json.dumps(payload, separators=(",", ":")).encode("utf-8"))
    res = lh.uploadBlob(blob, name, tag=tag)
    return res["data"]["Hash"]

cid = upload_json(
    {"agent_id": "researcher-7", "step": 1240, "tools": ["web", "rag"]},
    name="researcher-7-step-1240.json",
    tag="agent-state",
)

```

Upload a buffer (in-memory artifact)

```

python

import io
from lighthouseweb3 import Lighthouse

lh = Lighthouse(token=None)
buf = io.BytesIO(model.state_dict_serialized()) # bytes from pickle/safetensors
res = lh.uploadBlob(buf, "policy-net-v3.safetensors", tag="weights")
cid = res["data"]["Hash"]

```

The Python SDK's `download` returns `(bytes, meta)`. For streaming or browser-safe URLs, use the gateway directly.

```
python

content, meta = lh.download("Qm...")
open("restored.bin", "wb").write(content)

# Gateway URL pattern (premium accounts get gateway.lighthouse.storage; free
# accounts get a dedicated subdomain shown in the dashboard profile):
gateway_url = f"https://gateway.lighthouse.storage/ipfs/{cid}"
```

List user files and get file info

Both endpoints lack a Python SDK helper — call the REST API.

```
python

import os, requests

API = "https://api.lighthouse.storage"
HDRS = {"Authorization": f"Bearer {os.environ['LIGHTHOUSE_API_KEY']}" }

def list_uploads(last_key: str | None = None) -> dict:
    params = {"lastKey": last_key} if last_key else {}
    r = requests.get(f"{API}/api/user/files_uploaded", headers=HDRS, params=params,
    r.raise_for_status()
    return r.json()

def file_info(cid: str) -> dict:
    r = requests.get(f"{API}/api/lighthouse/file_info", headers=HDRS,
    params={"cid": cid}, timeout=30)
    r.raise_for_status()
    return r.json() # {fileName, fileSizeInBytes, encryption, mimeType, ...}
```

IPNS for mutable agent state pointers

IPNS lets a stable name resolve to whichever CID is current — exactly what you want for a "latest checkpoint" pointer that other agents follow without coordinating on CID changes. [Lighthouse](#)

```
python
```

```

import os, requests
API = "https://api.lighthouse.storage"
HDRS = {"Authorization": f"Bearer {os.environ['LIGHTHOUSE_API_KEY']}" }

def ipns_generate() -> dict:
    return requests.get(f"{API}/api/ipns/generate_key", headers=HDRS).json()
    # {"data": {"ipnsName": "...", "ipnsId": "k51qzi..."}}

def ipns_publish(cid: str, ipns_name: str) -> dict:
    return requests.get(
        f"{API}/api/ipns/publish_record",
        headers=HDRS, params={"cid": cid, "keyName": ipns_name},
    ).json()

# Pattern: one IPNS pointer per agent
pointer = ipns_generate()["data"]
ipns_publish(cid_of_latest_state, pointer["ipnsName"])
print(f"Mutable pointer: https://gateway.lighthouse.storage/ipns/{pointer['ipnsId']}")

```

Re-call `ipns_publish` with the same `ipnsName` and a new CID to **update** the pointer.

Lighthouse holds the IPNS signing key — no on-disk key management for mindX. `lighthouse`

Encrypted storage with Kavach

Kavach splits each file's AES-256 master key into **5 Shamir shards distributed across 5 independent nodes** under `encryption.lighthouse.storage`, with a **3-of-5 reconstruction threshold**. The plaintext key is never persisted anywhere. Access control is enforced cryptographically: a client must produce a wallet signature satisfying the file's access conditions before three nodes will release their shards. `Lighthouse + 5`

Because the Python SDK does not implement Kavach, mindX uses one of two patterns: **(A) shell out to a tiny Node helper** (most reliable, mirrors the maintained docs exactly), or **(B) call the encryption REST endpoints directly with `requests` + `eth_account` + `pycryptodome`**. Pattern A is recommended for production; pattern B is shown for completeness and for air-gapped deployments where the Node runtime is unavailable.

Auth message and JWT (Python)

```
python
```

```

import os, requests
from eth_account import Account
from eth_account.messages import encode_defunct

ENC = "https://encryption.lighthouse.storage"

def kavach_sign(privkey_hex: str) -> tuple[str, str]:
    """Returns (address, signedMessage_hex). Reusable for all Kavach calls."""
    acct = Account.from_key(privkey_hex)
    r = requests.get(f"{ENC}/api/message/{acct.address}", timeout=15)
    r.raise_for_status()
    payload = r.json()
    msg = payload[0]["message"] if isinstance(payload, list) else payload["data"]["m
    sig = acct.sign_message(encode_defunct(text=msg)).signature.hex()
    return acct.address, sig

```

For Algorand-derived agent identities (BANKON AlgoIDNFT), mindX should keep a **secondary EVM-compatible signing key** bound to the AlgoIDNFT via a BONAFIDE attestation, because Lighthouse's encryption auth flow expects ECDSA secp256k1 over EIP-191. The link `algorand_account ↔ ethereum_account` is recorded once in Tabularium so both rails accept the same agent identity.

Encrypted upload and decrypt — the Node-helper pattern (recommended)

Save this as `mindx/storage/kavach_helper.mjs`:

javascript

```

import lighthouse from "@lighthouse-web3/sdk";
const [op, ...args] = process.argv.slice(2);
const out = await ({
  upload: () => lighthouse.uploadEncrypted(args[0], args[1], args[2], args[3]),
  share: () => lighthouse.shareFile(args[0], JSON.parse(args[1]), args[2], args[3]),
  revoke: () => lighthouse.revokeFileAccess(args[0], JSON.parse(args[1]), args[2], a
  fetchKey: () => lighthouse.fetchEncryptionKey(args[0], args[1], args[2]),
  applyConditions: () => lighthouse.applyAccessCondition(
    args[0], args[1], args[2], JSON.parse(args[3]), args[4]),
}[op})();
process.stdout.write(JSON.stringify(out));

```

Drive it from Python:

python

```
import json, os, subprocess
from pathlib import Path

HELPER = Path(__file__).with_name("kavach_helper.mjs")

def _run(op: str, *args: str) -> dict:
    res = subprocess.run(
        ["node", str(HELPER), op, *args],
        capture_output=True, text=True, check=True, timeout=120,
    )
    return json.loads(res.stdout)

def upload_encrypted(path: str, address: str, signed: str) -> str:
    out = _run("upload", path, os.environ["LIGHTHOUSE_API_KEY"], address, signed)
    return out["data"][0]["Hash"]
```

Share access by wallet address

python

```
share = _run("share", owner_addr, json.dumps([recipient_addr]), cid, signed_msg)
# {'data': {'cid': '...', 'shareTo': ['0x...'], 'status': 'Success'}}
```

Token-gated access (BANKON AlgoIDNFT or BONAFIDE token)

`applyAccessCondition` lets mindX gate decryption on an on-chain check. The condition schema is documented for ERC-20, ERC-721, ERC-1155, native balances, custom contract calls, Solana SPL, and block-number (time-lock). For BANKON AlgoIDNFT-gated access, gate against the **EVM-side mirror contract** — the chain mapping at `agenticplace.pythai.net/allchain.html` declares which EVM chain the AlgoIDNFT has a canonical bridge on; pin that chain into the condition. (Astar)

python

```
# Decrypt only if the caller holds ≥1 BANKON AlgoIDNFT (EVM-mirror contract).
conditions = [{
    "id": 1,
    "chain": "Polygon", # chain per allchain.html mapping
    "method": "balanceOf",
    "standardContractType": "ERC721",
    "contractAddress": "0xBANKON_ALGOIDNFT_MIRROR_ADDRESS",
    "returnValueTest": {"comparator": ">=", "value": "1"},
    "parameters": [":userAddress"],
}, {
    "id": 2, # AND BONAFIDE reputation ≥ 50
    "chain": "Polygon",
    "method": "reputationOf",
    "standardContractType": "Custom",
    "contractAddress": "0xBONAFIDE_CONTRACT",
    "returnValueTest": {"comparator": ">=", "value": "50"},
    "parameters": [":userAddress"],
    "inputArrayType": ["address"],
    "outputType": "uint256",
}]
aggregator = "([1] and [2])"

_run("applyConditions", owner_addr, cid, signed_msg,
     json.dumps(conditions), aggregator)
```

For a **time-locked release** of a POD checkpoint (e.g., reveal the agent's pre-departure state only after block N on Optimism):

```
python

conditions = [{
    "id": 1, "chain": "Optimism", "method": "getBlockNumber",
    "standardContractType": "",
    "returnValueTest": {"comparator": ">", "value": str(target_block)},
}]
```

Solidity gating contracts (custom predicates beyond `balanceOf`) should be **Foundry-tested before deployment to mainnet**. A minimal Foundry pattern: write `test/AccessGate.t.sol`, fuzz `reputationOf`, `forge test --match-contract AccessGate -vv`, then `forge create --rpc-url $MAINNET_RPC --private-key $DEPLOY_KEY src/AccessGate.sol:AccessGate`.

Decrypt and retrieve

python

```
key_resp = _run("fetchKey", cid, address, signed_msg)
master_key_hex = key_resp["data"]["key"]    # 32-byte hex; raises if access denied

# Pull ciphertext from gateway, then AES-256-CBC decrypt (IV = first 16 bytes).
import requests
from Crypto.Cipher import AES
from Crypto.Util.Padding import unpad
ct = requests.get(f"https://gateway.lighthouse.storage/ipfs/{cid}", timeout=60).content
iv, body = ct[:16], ct[16:]
plaintext = unpad(AES.new(bytes.fromhex(master_key_hex), AES.MODE_CBC, iv).decrypt(body), 16)
```

Revoke

python

```
_run("revoke", owner_addr, json.dumps([revoked_addr]), cid, signed_msg)
```

After revocation, calls to `fetchEncryptionKey` from the revoked address return `{"message": "you don't have access", "data": {}}` and only ≤ 2 of 5 nodes will release shards — below the threshold. ([github](#))

Filecoin deal status and permanence

Every plaintext upload eventually gets sealed into Filecoin sectors via Lighthouse's aggregation pipeline; the docs warn this can take **a couple of hours up to ~1 day**. Until then `getDealStatus` returns an empty list. Once aggregated, you receive **deal IDs, storage providers (miners), epochs, and a PoDSI** (Proof of Data Segment Inclusion) proving that your sub-piece is contained in the SP's sealed deal. ([Lighthouse + 2](#))

python

```

from lighthouseweb3 import Lighthouse
import requests, time

lh = Lighthouse(token=None)

def deal_status(cid: str) -> list[dict]:
    resp = lh.getDealStatus(cid)
    return resp if isinstance(resp, list) else resp.get("data", [])

def podsi(cid: str, network: str = "mainnet") -> dict:
    r = requests.get(
        "https://api.lighthouse.storage/api/lighthouse/get_proof",
        params={"cid": cid, "network": network}, timeout=30,
    )
    r.raise_for_status()
    return r.json() # {pieceCID, dealInfo: [{dealId, storageProvider, ...}], proof:

def wait_for_filecoin(cid: str, timeout_s: int = 86_400, poll_s: int = 600) -> list[dict]:
    """Block until at least one Filecoin deal is recorded for the CID."""
    start = time.time()
    while time.time() - start < timeout_s:
        deals = deal_status(cid)
        if deals: return deals
        time.sleep(poll_s)
    raise TimeoutError(f"No Filecoin deal for {cid} after {timeout_s}s")

```

Two response shapes coexist across docs versions: a short `{DealID, Provider}` form and a long form with `dealId`, `storageProvider` (e.g. `f02620`), `pieceCID` (CommP, `baga6ea4...`), `payloadCid` (your IPFS CID), `pieceSize`, `endEpoch`, `dealStatus` (e.g. `"Sealing: PreCommit1"` → `"Active"`). **Code defensively against both.** (Lighthouse + 2)

For mindX, the rule is: **never claim "permanent" until `len(deal_status(cid)) >= 1` and at least one entry has `dealStatus in {"Active"}`** or the equivalent terminal state. Until then, the file is hot on IPFS but not yet endowment-backed. mindX should record `(cid, pieceCID, dealId, storageProvider, podsi_root)` into Tabularium so that BONAFIDE attestations can reference Filecoin-anchored truth, not just IPFS pin presence. **PDP** (Proof of Data Possession, the new 2025 hot-storage path) is queryable at `/api/lighthouse/pdp_deal_status?cid=...` for use cases that need faster-than-sealing durability proofs. (Lighthouse)

Agent storage patterns

Concrete patterns mindX should standardize across all agent classes.

Agent memory and checkpoint persistence

Every step (n) of an agent's loop, serialize state, upload as JSON, and update an IPNS pointer scoped to that agent. Other components dereference the IPNS name to get the latest state without coordination. **Idempotency is free**: identical state bytes always produce the same CID, so no extra storage is consumed when an agent's state hasn't changed.

```
python
```

```
def checkpoint_agent(agent_id: str, state: dict, ipns_name: str) -> str:
    cid = upload_json(state, f"{agent_id}-{state['step']}.json", tag=f"agent:{agent_id}")
    ipns_publish(cid, ipns_name)
    return cid
```

RAG corpus storage

Upload the corpus as a directory; the root CID is the **corpus version hash**. Pin the root CID into a BONAFIDE record so consumers verify provenance. For private corpora, use an encrypted per-file upload and apply token-gated access conditions referencing a BANKON AlgoIDNFT — only DID-holders can decrypt.

Shared agent knowledge bases with token-gated access

Use `(applyAccessCondition)` with a multi-condition aggregator — typically `((holds AlgoIDNFT) AND (BONAFIDE reputation  $\geq$  N))`. New agents that join the DAIO automatically gain read access by minting their BANKON identity; revocation is a single contract write, not a re-encryption.

POD checkpoints

Point of Departure checkpoints encode the immutable pre-launch state of an autonomous agent. Encrypt each POD with Kavach and apply a **time-lock condition** `((getBlockNumber > target))` on the chain selected per `(allchain.html)`. The CID + pieceCID + PoDSI tuple is then notarized into Tabularium and BONAFIDE so that, even after release, every observer can prove the POD existed at the original block time and was sealed onto Filecoin.

Cost accounting per agent

Tag every upload `((tag="agent:<id>"))`, then nightly call `(lh.getTagged(tag))` per agent and aggregate `(Size)`. Mirror this into the AgenticPlace DAIO's accounting ledger so budget enforcement (next section) has live numbers.

Integration with the wider stack

Lighthouse plugs into mindX's existing rails at four explicit seams.

x402 Algorand payment metering. Lighthouse documents a native **x402 pay-per-use upload** flow: the upload request carries an `X-PAYMENT` header containing a settled x402 micropayment, and no API key is required. mindX should extend this exact pattern across its **Parsec / parsec-wallet x402 Algorand rail**: agents create x402 payments against their own Algorand balances via parsec-wallet, mindX's storage proxy validates the payment, then forwards the upload to Lighthouse with a master API key. This makes **per-upload billing first-class** without exposing credentials to agents and without trusting agents to honor budgets. [Lighthouse](#)

BONAFIDE / Tabularium reputation pinning. Every important CID (POD checkpoint, DAIO decision artifact, model weight release) should be written into a Tabularium record. The BONAFIDE attestation contract stores `(cid, pieceCID, dealId, podsi_root)` and emits an event indexable by reputation aggregators. This turns Lighthouse storage into **on-chain provable storage** for any mindX-managed agent.

Chain selection from `allchain.html`. When applying access conditions, the chain on which BANKON, BONAFIDE, or any gating contract lives is authoritative per the AgenticPlace chain mapping. mindX should fetch `allchain.html` (or its JSON sibling) at startup, cache it, and pass `chain` into every `applyAccessCondition` call so deployments stay aligned across chain migrations.

Foundry testing for Solidity gating. Custom access predicates (anything beyond `balanceOf` / `getBlockNumber`) deploy as Solidity contracts. Use Foundry: `forge init`, write `test/AccessGate.t.sol`, fuzz with `forge test`, then deploy with `forge create` to mainnet only after green coverage. Lighthouse access conditions point to the deployed address via `contractAddress` — the same artifact mindX's agent registry references.

Agent-native MCP integration. Lighthouse ships an MCP server at `lighthouse-web3/lighthouse-agent-tooling` exposing tools `lighthouse_upload_file`, `lighthouse_create_dataset`, `lighthouse_fetch_file`. mindX-managed agents that already speak MCP can **register Lighthouse as an MCP tool provider** and skip the storage client entirely for routine operations. [GitHub](#) [lighthouse](#)

Pricing, quotas, and cost controls

Public plans as of April 2026 ([lighthouse.storage/pricing](#), monthly tab):

Plan	Price	Storage	Encryption / Token Gating	IPNS	Notes
Free Trial	\$0/mo	5 GB	Add-on	Add-on	Default for new accounts; uses dedicated gateway UR not <code>gateway.lighthouse.storage</code> <code>Lighthouse</code>
Lite	\$12/mo	500 GB	Add-on	Add-on	First production tier
Premium	\$49/mo	2.5 TB	Add-on	Add-on	Unlocks <code>gateway.lighthouse.storage</code> <code>Lighthouse</code>
Custom / Lifetime	Contact	—	—	—	Lifetime tier and per-deal endowment-pool deposits available
Pay-per-deal	Variable	Per upload	n/a	n/a	Endowment Pool address <code>0x0E8b07CefDC0363cA6e0Ca06093c2596746f74</code> <code>Lighthouse</code> priced via <code>lighthouse.getQuote</code>
x402 Pay-Per-Use	Per upload	Per upload	n/a	n/a	Settled per-upload via x402 (Base/baseSepolia in doc mindX bridges to Algorand)

Lighthouse no longer publishes a flat per-GB perpetual rate in the new docs; **for guaranteed numbers, query `lighthouse.getQuote()` at upload time** or request enterprise pricing. Bandwidth from `gateway.lighthouse.storage` is gated to paid users; free-tier accounts receive a dedicated gateway subdomain shown in the dashboard profile. `Lighthouse`

For per-agent budget enforcement, mindX should implement a soft-cap pattern:

```
python
```

```
def assert_budget(agent_id: str, monthly_byte_cap: int) -> None:
    used = sum(int(f["fileSizeInBytes"]) for f in lh.getTagged(f"agent:{agent_id}"))
    if used >= monthly_byte_cap:
        raise BudgetExceeded(f"{agent_id} used {used} ≥ cap {monthly_byte_cap}")
```

Combine this with a hard cap enforced by **x402 escrow per agent** at the Parsec layer — even if the soft cap fails open, the agent's wallet runs out of x402 micropayments and Lighthouse refuses uploads. This two-layer pattern (off-chain accounting + on-chain escrow) prevents runaway spend from a misbehaving agent.

Error handling and reliability

Lighthouse does **not publish formal rate limits or a retry policy**. Practical guidance distilled from SDK behavior and ecosystem norms:

mindX should treat uploads as **idempotent on the CID** — re-uploading identical bytes returns the same `Hash` and is safe to retry blindly. Use exponential backoff (start 1s, factor 2, max 60s, jitter) on any 5xx or transport error. For 401 responses, immediately stop retries and raise — the API key is wrong or revoked. For 4xx other than 401, log and stop. Filecoin deal queries should poll at 10–60 minute intervals, not sub-minute, since sealing is a multi-hour process. The 5-node Kavach network is itself fault-tolerant via 3-of-5 — a single node outage is invisible to clients, but two simultaneous outages start to fail decryption; observe this in metrics and route around it by pre-warming JWTs on healthy nodes.

For retrieval, build a **gateway fallback chain**: try the dedicated Lighthouse gateway, then `gateway.lighthouse.storage` (if you're premium), then public IPFS gateways (`ipfs.io`, `dweb.link`, `cloudflare-ipfs.com`). Plain (non-encrypted) CIDs propagate to the public DHT and are reachable from any IPFS node, which is mindX's resilience floor.

The maximum single-upload size is **24 GB**; chunk any larger artifact into a directory upload or split at the application layer with a manifest CID stitching the parts. `Lighthouse`

Security checklist

Do: store `LIGHTHOUSE_API_KEY` in environment variables only; rotate keys per agent class; use the JWT method for repeated Kavach calls and keep JWTs in memory only; sign Kavach messages **server-side** so private keys never touch the agent runtime; validate every `applyAccessCondition` response on a Foundry-tested contract before going to mainnet; record every important CID into BONAFIDE / Tabularium with its PoDSI; verify `dealStatus` before claiming permanence to users; pin the chain name from `allchain.html` rather than hardcoding.

Don't: bake API keys into Podman images or git commits; expose API keys to browser-side code (use a server proxy); reuse the same wallet across encryption owner / API-key generator / x402 payer (split duties); rely on a single Kavach node — always require 3-of-5; trust IPFS pinning alone for "permanence" (must have Filecoin deals); treat the unverified `node.lighthouse.storage` host as canonical (use `upload.lighthouse.storage`); skip TLS verification on `requests` calls; share Kavach JWTs between agents.

Reference: API endpoints and SDK methods

Method / Path	Purpose	Auth
<code>GET api.lighthouse.storage/api/auth/get_message?publicKey=</code>	Auth nonce for API-key creation	none
<code>POST api.lighthouse.storage/api/auth/create_api_key</code>	Create scoped API key	sig in body
<code>POST upload.lighthouse.storage/api/v0/add</code>	Upload file/dir (multipart)	Bearer API key
<code>GET gateway.lighthouse.storage/ipfs/{cid}</code>	Retrieve plaintext CID	none
<code>GET gateway.lighthouse.storage/ipfs/{cid}?h=&w=</code>	On-the-fly image resize	none
<code>GET gateway.lighthouse.storage/ipns/{ipnsId}</code>	Resolve IPNS pointer	none
<code>GET api.lighthouse.storage/api/lighthouse/file_info?cid=</code>	File metadata	Bearer
<code>GET api.lighthouse.storage/api/user/files_uploaded</code>	List uploads (paginated by <code>lastKey</code>)	Bearer
<code>GET api.lighthouse.storage/api/user/user_data_usage</code>	Quota / used bytes	Bearer
<code>GET api.lighthouse.storage/api/lighthouse/deal_status?cid=</code>	Filecoin deal status	none/Bearer
<code>GET api.lighthouse.storage/api/lighthouse/get_proof?cid=&network=</code>	PoDSI proof	none
<code>GET api.lighthouse.storage/api/lighthouse/pdp_deal_status?cid=</code>	PDP hot-storage deal status	none
<code>GET api.lighthouse.storage/api/lighthouse/pdp_deal_request?cid=</code>	PDP request details	Bearer

Method / Path	Purpose	Auth
GET <code>api.lighthouse.storage/api/ipns/generate_key</code>	New IPNS key	Bearer
GET <code>api.lighthouse.storage/api/ipns/publish_record?cid=&keyName=</code>	Publish/update IPNS	Bearer
GET <code>api.lighthouse.storage/api/ipns/get_ipns_records</code>	List IPNS records	Bearer
GET <code>api.lighthouse.storage/api/ipns/remove_key?keyName=</code>	Delete IPNS key	Bearer
GET <code>encryption.lighthouse.storage/api/message/{addr}</code>	Kavach auth nonce	none
(Kavach REST, per node 1..5)	save_shards / recover_shards / share / revoke / setAccessConditions / verifyZkConditions	signed-r / JWT
POST <code>{x402_endpoint}/api/x402/upload</code>	Pay-per-use upload (x402 settle)	x402 hea

Python (`lighthouseweb3`) (v0.1.6) supported methods: `Lighthouse(token)`,
`lh.upload(source, tag=)`, `lh.uploadBlob(file_obj, name, tag="")`,
`lh.getUploads(cid)`, `lh.download(cid)`, `lh.getDealStatus(cid)`, `lh.getTagged(tag)`.
Everything else → HTTP or Node helper.

Node (`@lighthouse-web3/sdk`) (v0.4.1) full surface: `upload`, `uploadBuffer`, `uploadText`,
`uploadEncrypted`, `textUploadEncrypted`, `getFileInfo`, `getUploads`, `getBalance`,
`dealStatus`, `getAuthMessage`, `getApiKey`, `generateKey`, `publishRecord`, `getAllKeys`,
`removeKey`, `shareFile`, `revokeFileAccess`, `applyAccessCondition`, `getAccessConditions`,
`fetchEncryptionKey`, `decryptFile`.

CLI (`lighthouse-web3`, installed with the JS SDK): `wallet`, `create-wallet`, `import-wallet`,
`balance`, `upload`, `upload-encrypted`, `decrypt-file`, `share-file`, `revoke-access`, `deal-`
`status`, `get-uploads`, `api-key --new`, `ipns --generate-key|--publish|--list|--remove`.
Go alternative: `lhctl` with equivalent flag-driven syntax. `Lighthouse` `lighthouse`

Appendix: Minimal mindX Lighthouse client

Drop straight into `mindx/storage/lighthouse_client.py`. Production-ready, env-driven, fully

typed, with explicit exception classes and docstrings. Kavach features delegate to a sibling `kavach_helper.mjs` (shown in the encrypted-storage section).

```
python
```

```
"""mindx.storage.lighthouse_client - Lighthouse Storage adapter for mindX.
```

```
Exposes a stable, typed interface over the Lighthouse Python SDK + REST API  
+ a Node helper for Kavach features the Python SDK does not implement.
```

```
Env:
```

```
    LIGHTHOUSE_API_KEY    dashboard API key (required for all writes)  
    LIGHTHOUSE_GATEWAY    override gateway base (default gateway.lighthouse.storage)  
    MINDX_KAVACH_HELPER    path to kavach_helper.mjs (default sibling of this file)
```

```
"""
```

```
from __future__ import annotations
```

```
import io
```

```
import json
```

```
import os
```

```
import subprocess
```

```
import time
```

```
from dataclasses import dataclass
```

```
from pathlib import Path
```

```
from typing import Any, Iterable
```

```
import requests
```

```
from lighthouseweb3 import Lighthouse
```

```
# ----- exceptions -----
```

```
class LighthouseError(Exception):
```

```
    """Base class for all Lighthouse client errors."""
```

```
class AuthError(LighthouseError):
```

```
    """Missing or invalid LIGHTHOUSE_API_KEY / Kavach signature."""
```

```
class UploadError(LighthouseError):
```

```
    """Upload failed after retries."""
```

```
class AccessDenied(LighthouseError):
```

```
    """Kavach refused to release shards (caller fails access conditions)."""
```

```
class DealNotReady(LighthouseError):
```

```
    """No Filecoin deal yet for this CID."""
```

```
# ----- types -----
```

```
@dataclass(frozen=True)
```

```
class Upload:
```

```

    cid: str
    name: str
    size: int

@dataclass(frozen=True)
class Deal:
    deal_id: int
    storage_provider: str
    piece_cid: str | None
    status: str | None

# ----- client -----
class LighthouseClient:
    """Production Lighthouse adapter. Thread-safe; cheap to instantiate."""

    API = "https://api.lighthouse.storage"
    ENC = "https://encryption.lighthouse.storage"

    def __init__(self, api_key: str | None = None, *, gateway: str | None = None,
                  helper_path: str | os.PathLike | None = None, timeout: int = 60) -> None:
        key = api_key or os.getenv("LIGHTHOUSE_API_KEY")
        if not key:
            raise AuthError("LIGHTHOUSE_API_KEY not set")
        self._key = key
        self._lh = Lighthouse(token=key)
        self._gateway = gateway or os.getenv(
            "LIGHTHOUSE_GATEWAY", "https://gateway.lighthouse.storage")
        self._helper = Path(helper_path or os.getenv(
            "MINDX_KAVACH_HELPER",
            Path(__file__).with_name("kavach_helper.mjs")))
        self._timeout = timeout

    # ---- auth headers ----
    @property
    def _hdrs(self) -> dict[str, str]:
        return {"Authorization": f"Bearer {self._key}"}

    # ---- plaintext ops ----
    def upload_file(self, path: str | os.PathLike, *, tag: str = "") -> Upload:
        """Upload a single file or directory by path. Idempotent on bytes."""
        try:
            r = self._lh.upload(source=str(path), tag=tag) if tag \
                else self._lh.upload(source=str(path))
        except Exception as e:

```

```

        raise UploadError(f"upload failed: {e}") from e
    d = r["data"]
    return Upload(cid=d["Hash"], name=d["Name"], size=int(d["Size"]))

def upload_bytes(self, blob: bytes, name: str, *, tag: str = "") -> Upload:
    """Upload an in-memory byte buffer (model weights, serialized state)."""
    try:
        r = self._lh.uploadBlob(io.BytesIO(blob), name, tag=tag)
    except Exception as e:
        # noqa: BLE001
        raise UploadError(f"uploadBlob failed: {e}") from e
    d = r["data"]
    return Upload(cid=d["Hash"], name=d["Name"], size=int(d["Size"]))

def upload_json(self, payload: Any, name: str, *, tag: str = "") -> Upload:
    """Upload a JSON-serializable object. Canonical for agent state."""
    return self.upload_bytes(
        json.dumps(payload, separators=(",", ":"), sort_keys=True).encode("utf-8"),
        name, tag=tag,
    )

# ---- retrieval ----
def retrieve(self, cid: str) -> bytes:
    """Fetch raw bytes for a plaintext CID via the gateway."""
    r = requests.get(f"{self._gateway}/ipfs/{cid}", timeout=self._timeout)
    if r.status_code == 404:
        raise LighthouseError(f"CID not found: {cid}")
    r.raise_for_status()
    return r.content

def gateway_url(self, cid: str) -> str:
    return f"{self._gateway}/ipfs/{cid}"

# ---- listing & info ----
def list_files(self, *, last_key: str | None = None) -> dict:
    """Paginated list of all files under this API key."""
    params = {"lastKey": last_key} if last_key else {}
    r = requests.get(f"{self.API}/api/user/files_uploaded",
                     headers=self._hdrs, params=params, timeout=self._timeout)
    r.raise_for_status()
    return r.json()

def file_info(self, cid: str) -> dict:
    r = requests.get(f"{self.API}/api/lighthouse/file_info",
                     headers=self._hdrs, params={"cid": cid}, timeout=self._time

```

```

        r.raise_for_status()
        return r.json()

def usage(self) -> dict:
    """Bytes used / limit for the current account."""
    r = requests.get(f"{self.API}/api/user/user_data_usage",
                     headers=self._hdrs, timeout=self._timeout)
    r.raise_for_status()
    return r.json()

# ---- Filecoin permanence ----
def deal_status(self, cid: str) -> list[Deal]:
    """Return list of Filecoin deals; empty until aggregation completes."""
    raw = self._lh.getDealStatus(cid)
    items: Iterable[dict] = raw if isinstance(raw, list) else raw.get("data", [])
    out: list[Deal] = []
    for it in items:
        out.append(Deal(
            deal_id=int(it.get("dealId") or it.get("DealID") or 0),
            storage_provider=str(it.get("storageProvider") or
                                   it.get("miner") or it.get("Provider") or ""),
            piece_cid=it.get("pieceCID"),
            status=it.get("dealStatus"),
        ))
    return out

def podsi(self, cid: str, *, network: str = "mainnet") -> dict:
    r = requests.get(f"{self.API}/api/lighthouse/get_proof",
                     headers=self._hdrs,
                     params={"cid": cid, "network": network},
                     timeout=self._timeout)
    r.raise_for_status()
    return r.json()

def wait_for_filecoin(self, cid: str, *, timeout_s: int = 86_400,
                      poll_s: int = 600) -> list[Deal]:
    start = time.time()
    while time.time() - start < timeout_s:
        deals = self.deal_status(cid)
        if deals:
            return deals
        time.sleep(poll_s)
    raise DealNotReady(f"no Filecoin deal for {cid} after {timeout_s}s")

```

```

# ---- IPNS (mutable agent pointers) ----
def ipns_generate(self) -> dict:
    r = requests.get(f"{self.API}/api/ipns/generate_key",
                     headers=self._hdrs, timeout=self._timeout)
    r.raise_for_status(); return r.json()["data"]

def ipns_publish(self, cid: str, ipns_name: str) -> dict:
    r = requests.get(f"{self.API}/api/ipns/publish_record",
                     headers=self._hdrs,
                     params={"cid": cid, "keyName": ipns_name},
                     timeout=self._timeout)
    r.raise_for_status(); return r.json()

# ---- Kavach (delegated to Node helper) ----
def _node(self, op: str, *args: str) -> dict:
    if not self._helper.exists():
        raise LighthouseError(f"kavach helper not found: {self._helper}")
    try:
        res = subprocess.run(
            ["node", str(self._helper), op, *args],
            capture_output=True, text=True, check=True, timeout=180)
    except subprocess.CalledProcessError as e:
        if "access" in (e.stderr or "").lower():
            raise AccessDenied(e.stderr) from e
        raise LighthouseError(f"kavach helper {op} failed: {e.stderr}") from e
    return json.loads(res.stdout or "{}")

def upload_encrypted(self, path: str | os.PathLike,
                     address: str, signed_message: str) -> Upload:
    out = self._node("upload", str(path), self._key, address, signed_message)
    d = (out.get("data") or [{}])[0]
    if not d.get("Hash"):
        raise UploadError(f"encrypted upload returned no CID: {out}")
    return Upload(cid=d["Hash"], name=d.get("Name", ""), size=int(d.get("Size", 0)))

def share(self, owner: str, cid: str, signed_message: str,
           recipients: list[str]) -> dict:
    return self._node("share", owner, json.dumps(recipients), cid, signed_message)

def revoke(self, owner: str, cid: str, signed_message: str,
            revoke_from: list[str]) -> dict:
    return self._node("revoke", owner, json.dumps(revoke_from), cid, signed_message)

def apply_conditions(self, owner: str, cid: str, signed_message: str,

```

```

        conditions: list[dict], aggregator: str) -> dict:
    return self._node("applyConditions", owner, cid, signed_message,
                      json.dumps(conditions), aggregator)

def retrieve_decrypted(self, cid: str, address: str,
                      signed_message: str) -> bytes:
    out = self._node("fetchKey", cid, address, signed_message)
    key_hex = (out.get("data") or {}).get("key")
    if not key_hex:
        raise AccessDenied(out.get("message") or "access denied")
    from Crypto.Cipher import AES
    from Crypto.Util.Padding import unpad
    ct = self.retrieve(cid)
    iv, body = ct[:16], ct[16:]
    return unpad(AES.new(bytes.fromhex(key_hex), AES.MODE_CBC, iv).decrypt(body))

__all__ = [
    "LighthouseClient", "Upload", "Deal",
    "LighthouseError", "AuthError", "UploadError",
    "AccessDenied", "DealNotReady",
]

```

Drop-in usage from anywhere in mindX:

```

python

from mindx.storage.lighthouse_client import LighthouseClient

lh = LighthouseClient() # reads env
state_cid = lh.upload_json({"step": 1, "ok": True}, "agent-7.json",
                           tag="agent:7").cid
deals = lh.wait_for_filecoin(state_cid, timeout_s=3600, poll_s=300)

```

Conclusion

Lighthouse is the right storage substrate for mindX because three of its protocol-level properties map directly onto the AgenticPlace stack: **content-addressed idempotency** matches BANKON / BONAFIDE's hash-anchored attestations; **Kavach 3-of-5 threshold encryption with on-chain access conditions** lets mindX enforce DAIO membership cryptographically against the chains catalogued at [allchain.html](#); and **endowment-pool-funded perpetual Filecoin deals with**

PoDSI turn every agent artifact into an audit-grade cold record without recurring rent. The integration cost is small but real — the Python SDK stops at uploads, so the production client must speak HTTP for IPNS/listing and shell to a Node helper for Kavach. The minimal client in the appendix absorbs this complexity behind a clean, typed interface, and the x402 metering pattern wires Lighthouse spending into Parsec's Algorand rails so agent budgets are enforced at the wallet layer rather than on trust. Mounted into Podman with a per-agent-class API key and an `agent:<id>` tagging convention, Lighthouse becomes mindX's permanent memory — verifiable, gateable, and cheap to read.